Document number: P1069R0
Date: 2018-10-08
Reply-To:
Mike Spertus, Symantec (mike_spertus@symantec.com)
Walter E. Brown (webrown.cpp@gmail.com)
Stephan T. Lavavej (stl@exchange.microsoft.com)
Audience: {Library Evolution, Library} Working Group

Refining standard library support for Class Template Argument Deduction

Introduction

In this paper, we describe the changes to library support for class template argument deduction that we believe are appropriate for C++20.

- Adding new overloads to `make_unique` and `make_shared` to support an important use case
- (Re)adding some container constructors in support of P1021R1, which (if adopted) we hope will provide a powerful use case for supporting deduction from allocators that did not exist in C++17.
- Finally, while we think the committee did an exceptional job on standard library deduction guides in C++17 and there is very little that we would have changed in retrospect except for one decision (the handling of `reference_wrappers`) that we feel should be corrected. We present a rationale that we believe was not discussed at the time in support of this

Overloading `make_unique` and `make_shared`

While class template argument deduction can be used for objects with static and automatic duration, it generally cannot be used for creating objects with dynamic duration as the following attempts show:

```
optional o(5); // OK. Static duration. optional<int>

// Try to create dynamic duration object.
auto o1 = new optional(5);  // Already violates both Core Guidelines R.3 and R.11

auto o2 = unique_ptr(o1); // Oops, ill-formed. Can't deduce unique_ptr from raw ptr
```

We propose simply making dynamic object creation with `make_unique` and `make_shared` work just like the normal way one would translate a static and automatic declaration to a dynamic declaration

```
int i1(5);                      // Static
auto i2 = make_unique<int>(5);   // Make dynamic by moving decl-specifier to make_unique template argument
option o1(5);
auto o2 = make_unique<optional>(5); // Again, make dynamic by moving decl-specifier to make_unique template argument
```

Implementing this can be as simple as the following code, which can be seen working on Wandbox

```
template<template <typename ...U> typename T, typename ...A>
auto make_unique(A&& ...a) {
    return std::unique_ptr<decltype(T(std::forward<A>(a)...))>(new T(std::forward<A>(a)...));
}
```

Notes:

- For simplicity, we do not propose supporting the case of non-type template parameters as it is far less frequent and seems far more complex. If and only if library implementers feel that there are no implementation concerns for non-type template parameters, then that should be supported as well
- It is worth pointing out that because this code exactly matches *mutatis mutandi* the automatic duration declaration case, it should have no cause to be in any way more dangerous or confusing that what is supported for objects created with static or automatic duration

Allow containers to deduce from allocators or comparators

Note: This item has a dependency on P1021R1 as described below.

A number of associative container guides were removed in the late stages of the Kona meeting for the following reasons

- Since the `size_type` member may or may not be a deduced context, and different compilers handle “most specialized” differently when some candidates are deducible and others are non-deducible for the same argument, there were implementability concerns.
- Even where there were no implementability concerns, construction from allocators was removed from all other containers for consistency with associative containers.
- A general agreement that trying to deduce the value type for a container from its allocator was an anti-pattern
- In addition, there were no constructors from comparators because the value type could not be deduced from a comparator anyway.

We propose restoring guides so that containers deduce consistently from all of their constructors not because we disagree with any of the above, but because P1021R1's support for partial specialization in class template argument deduction, if accepted, adds a very important and useful new use case for them.

```
vector<int> v{MyAlloc{}}; // Want vector<int, MyAlloc>
set<string> caseInsensitiveStrings([](string const &a, string const &b) { /* ... */ });
```

Speaking “conceptually”, we also suggest that deducing correctly from all constructors is valuable because the reason that STL containers need a lot of deduction guides in the first place is because they are not conceptized (See the bottom of http://qr.w69b.com/g/qE1JDD0Sk for a demonstration of some current deduction guides becoming unnecessary in a conceptized STL). If we ever have a concept-enabled STL, then the example in use case 1 above will work even without a deduction guide, so this is not only useful now but better prepares us for the future concepts vision.

The new deduction guides are similar to the previously removed deduction guides with the following changes, which incidentally greatly simplify their original versions:

- To steer clear of the technical concern mention above, we simply use `size_t` in the following wording (Note that using `size_t` does not assume that the container's `size_type` is `size_t`, just that it is just an integral type being used for overload resolution).
- We do not attempt to deduce the value type from the allocator. This is sufficient for the new use cases and steers clear of the above concern
- We also add deduction from comparators

`reference_wrapper` and `pair/tuple` deduction

In P0433R0, it was proposed that deduction guides for `pair` and `tuple` unwrap `reference_wrapper` like `make_pair` and `make_tuple` do. Such unwrapping was removed from later revisions of the paper and not adopted in C++17. According to both the minutes and the mailing list, unwrapping was suppressed in order to increase applicability to fully generic programming. However, as the following code available at https://wandbox.org/permlink/rRITtZ2gT2ZqR4jY shows, the possibility of picking up stray constructors suggests that fully generic programming should fully specialize template parameters anyway (note that this best practice is not limited to CTAD) rather than relying on `tuple` CTAD.

```
template<typename ...T>  // CTAD: Intends tuple<T...> but stray constructors make unreliable
using ctad = decltype(tuple{declval<T>()...});

template<typename ...T>  // Explicit:  Produces tuple<T...> as intended
using expl = decltype(tuple<T...>{declval<T>()...});

int main()
{
    print_type<ctad<tuple<lt;int>>>();  // std::tuple<int>
    print_type<expl<tuple<int>>>();  // std::tuple<std::tuple<int>>
    print_type<ctad<allocator_arg_t, allocator<int>, int>>();  // std::tuple<int>
    print_type<expl<allocator_arg_t, allocator<int>, int>>();  // std::tuple<std::allocator_arg_t, std::allocator<int>, int>
    return 0;
}
```

However, with the C++17 behavior, not only does `pair/tuple` deduction not support generic metaprogramming, its inability to deduce references limits its ability to replace `make_tuple` or `make_pair` even though many expositions (E.g., [1], [2], and [3]) give `make_pair` replacement as their motivating use case. As library vocabulary vocabulary types whose pre-CTAD factory functions unwrap `reference_wrapper`, we suggest that behavior preserved in CTAD.

Wording

In addition to the below, change all of the “*see below*::size_type” with `size_t` in deduction guides, simplifying and increasing robustness

Add the following deduction guide to the definition of class `basic_string` in §24.3.2 [basic.string]:

```
      int compare(size_type pos1, size_type n1,
                  const charT* s, size_type n2) const;
    };

  template<class T, class Traits = char_traits<T>, class Allocator>
  basic_string(Allocator) -> basic_string<T, Traits, Allocator>;
```

In §24.3.2.2 [string.cons], insert the following:

```
  template<class T,  class Traits = char_traits<T>, class Allocator>
  basic_string(Allocator) -> basic_string<T, Traits, Allocator>;
```

> *Remarks:* Shall not participate in overload resolution if `Allocator` is a type that does not qualify as an allocator [sequence.reqmts].

At the end of the definition of class `deque` in §26.3.8.1 [deque.overview], add the following deduction guides:

```
    void clear() noexcept;
  };

  template<class T, class Allocator>
  explicit deque(Allocator) -> deque<T, Allocator>;

  template<class T, class Allocator> explicit deque(size_t, Allocator) -> deque<T, Allocator>;
```

At the end of the definition of class `forward_list` in §26.3.9.1 [forwardlist.overview], add the following deduction guides:

```
  void reverse() noexcept;
};

template<class T, class Allocator>
explicit forward_list(Allocator) -> forward_list<T, Allocator>;

template<class T, class Allocator>
explicit forward_list(size_t, Allocator) -> forward_list<T, Allocator>;
```

At the end of the definition of class `list` in §26.3.10.1 [list.overview], add the following deduction guides:

```
  void reverse() noexcept;
};

template<class T, class Allocator>
explicit list(Allocator) -> list<T, Allocator>;

template<class T, class Allocator>
explicit list(size_t, Allocator) -> list<T, Allocator>;
```

At the end of the definition of class `vector` in §26.3.11.1 [vector.overview], add the following *deduction-guide*:

```
  void clear() noexcept;
};

template<class T, class Allocator>
explicit  vector(Allocator) -> vector<T, Allocator>;

template<class T, class Allocator>
explicit  vector(size_t, Allocator)
  -> vector<T, Allocator>;
```

At the end of the definition of class `map` in §26.4.4.1 [map.overview], add the following deduction guides:

```
  pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template<class Key, class T, class Compare, class Allocator = allocator<pair<Key, T>>>
explicit map(Compare, Allocator = Allocator())
  -> map<Key, T, Compare, Allocator>;

template<class Key, class T, class Compare = less<Key>, class Allocator>
explicit map(Allocator)
  -> map<Key, T, Compare, Allocator>;
```

At the end of the definition of class `multimap` in §26.4.5.1 [multimap.overview], add the following deduction guides:

```
  pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template<class Key, class T, class Compare, class Allocator = allocator<pair<Key, T>>>
explicit multimap(Compare, Allocator = Allocator())
  -> multimap<Key, T, Compare, Allocator>;

template<class Key, class T, class Compare = less<Key>, class Allocator>
explicit multimap(Allocator)
  -> multimap<Key, T, Compare, Allocator>;
```

At the end of the definition of class `set` in §26.4.6.1 [set.overview], add the following *deduction-guide*s:

```
  template <class K>
    pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template<class T, class Compare, class Allocator>
explicit set(Compare, Allocator = allocator<T>)
  -> set<T, Compare, Allocator>;


template<class T, class Compare = less<T>, class Allocator>
explicit set(Allocator)
  -> set<T, Compare, Allocator>;
```

At the end of the definition of class `multiset` in §27.4.6.1 [multiset.overview], add the following deduction guides:

```
  template <class K>
    pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template<class T, class Compare, class Allocator>
explicit multiset(Compare, Allocator = allocator<T>)
  -> multiset<T, Compare, Allocator>;


template<class T, class Compare = less<T>, class Allocator>
```

```
  explicit multiset(Allocator)
    -> multiset<T, Compare, Allocator>;
```

Modify §26.5.4.1 [unord.map.overview], add the following *deduction-guide*s:

```
    void reserve(size_type n);
  };

  template<class Key, class T, class Hash = hash<Key>, class Pred = equal_to<Key>, class Allocator>
  explicit unordered_map(size_t, Hash, Pred = Pred(), Allocator = Allocator())
      -> unordered_map<Key, T, Hash, Pred, Allocator>;

  template<class Key, class T, class Hash = hash<Key>, class Pred = equal_to<Key>, class Allocator>
  explicit unordered_map(Allocator)
    -> unordered_map<Key, T, Hash, Pred, Allocator>;


  template<class Key, class T, class Hash = hash<Key>, class Pred = equal_to<Key>, class Allocator>
  explicit unordered_map(size_t, Allocator)
      -> unordered_map<Key, T, Hash, Pred, Allocator>;

  template<class Key, class T, class Hash, class Pred = equal_to<Key>, class Allocator>
  explicit unordered_map(size_t, Hash, Allocator)
    -> unordered_map<Key, T,
                Hash, Pred, Allocator>;
```

Delete §26.5.4.1p4 [unord.map.overview]:

~~A `size_type` parameter type in an `unordered_map` deduction guide refers to the `size_type` member type of the type deduced by the deduction guide.~~

Modify §26.5.5.1 [unord.multimap.overview], add the following *deduction-guide*s:

```
    void reserve(size_type n);
  };

  template<class Key, class T, class Hash = hash<Key>, class Pred = equal_to<Key>, class Allocator>
  explicit unordered_multimap(size_t, Hash, Pred = Pred(), Allocator = Allocator())
      -> unordered_multimap<Key, T, Hash, Pred, Allocator>;

  template<class Key, class T, class Hash = hash<Key>, class Pred = equal_to<Key>, class Allocator>
  explicit unordered_multimap(Allocator)
    -> unordered_multimap<Key, T, Hash, Pred, Allocator>;


  template<class Key, class T, class Hash = hash<Key>, class Pred = equal_to<Key>, class Allocator>
  explicit unordered_multimap(size_t, Allocator)
      -> unordered_multimap<Key, T, Hash, Pred, Allocator>;

  template<class Key, class T, class Hash, class Pred = equal_to<Key>, class Allocator>
  explicit unordered_multimap(size_t, Hash, Allocator)
    -> unordered_multimap<Key, T,
                Hash, Pred, Allocator>;
```

Delete §26.5.5.1p4 [unord.multimap.overview]:

~~A `size_type` parameter type in an `unordered_multimap` deduction guide refers to the `size_type` member type of the type deduced by the deduction guide.~~

Modify §26.5.6.1 [unord.set.overview] as follows:

```
    void reserve(size_type n);
  };

  template<class T, class Hash, class Pred = equal_to<T>, class Allocator = allocator<T>>
  explicit unordered_set(size_t, Hash, Pred = Pred(), Allocator = Allocator())
    -> unordered_set<T, Hash, Pred, Allocator>;


  template<class T, class Hash = hash<T>, Pred = equal_to<T>, class Allocator>
  explicit unordered_set(size_t, Allocator)
    -> unordered_set<T, Hash, Pred, Allocator>;

  template<class T, class Hash, Pred = equal_to<T>, class Allocator>
  explicit unordered_set(size_t, Hash, Allocator)
    -> unordered_set<T, Hash, Pred, Allocator>;
```

Delete §26.5.6.1p4 [unord.set.overview]:

~~A `size_type` parameter type in an `unordered_set` deduction guide refers to the `size_type` member type of the primary `unordered_set` template.~~

Modify §26.5.7.1 [unord.multiset.overview] as follows:

```
    void reserve(size_type n);
  };

  template<class T, class Hash, class Pred = equal_to<T>, class Allocator = allocator<T>>
  explicit unordered_multiset(size_t, Hash, Pred = Pred(), Allocator = Allocator())
    -> unordered_multiset<T, Hash, Pred, Allocator>;


  template<class T, class Hash = hash<T>, Pred = equal_to<T>, class Allocator>
  explicit unordered_multiset(size_t, Allocator)
    -> unordered_multiset<T, Hash, Pred, Allocator>;

  template<class T, class Hash, Pred = equal_to<T>, class Allocator>
  explicit unordered_multiset(size_t, Hash, Allocator)
    -> unordered_multiset<T, Hash, Pred, Allocator>;
```

Delete §26.5.7.1p4 [unord.multiset.overview]:

~~A `size_type` parameter type in an `unordered_multiset` deduction guide refers to the `size_type` member type of the primary `unordered_multiset` template.~~

At the end of the definition of class `promise` in §33.6.6 [futures.promise], insert the following:

```
    // setting the result with deferred notification
    void set_value_at_thread_exit(see below);
    void set_exception_at_thread_exit(exception_ptr p);
  };

  template <class T, class Alloc> promise(allocator_arg_t, Alloc)
    -> promise<T>;

  template <class R>
    void  swap(promise<R>& x,  promise<R>& y) noexcept;
```